

Implementation of:- Containerized Application Deployment using Docker, ECR, and ECS

Objective of the Project

The objective is to show how a web application can be built locally, containerized with Docker, stored in AWS ECR, and deployed seamlessly on ECS with automatic scaling and monitoring — making it production-ready in the cloud.

1. Develop the App

- **Build the Node.js air-ticketing application locally.**
- **Test endpoints to confirm functionality.**

2. Containerize with Docker

- Create a Dockerfile and build the image.
- Run locally to ensure portability and consistency.

3. Push Image to Amazon ECR

- **Tag the Docker image with your AWS account repository.**
- **Push it securely to ECR for storage and version control.**

4. Deploy with Amazon ECS

- Use Express Mode to select the image from ECR.
- ECS provisions tasks, services, and networking automatically.

5. Networking & Scaling

- **ECS integrates with Load Balancer, Security Groups, and Auto Scaling.**
- **Ensures high availability, controlled access, and performance.**

6. Application Access

- ECS generates a public Application URL.
- Users can access the air-ticketing app live in the cloud.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\JAY> mkdir air-ticketing-app

Directory: C:\Users\JAY

Mode                LastWriteTime         Length Name
----                -
d-----          05-02-2026    16:14         air-ticketing-app

PS C:\Users\JAY> |
```

- This created a new folder named air-ticketing-app under .
- The terminal confirms the directory details like mode, last write time, and name.

```

PS C:\Users\JAY\air-ticketing-app> New-Item -ItemType File -Path index.html, style.css, package.json, server.js, Dockerfile,
docker-compose.yml, ecs-task-def.json, app.js -Force

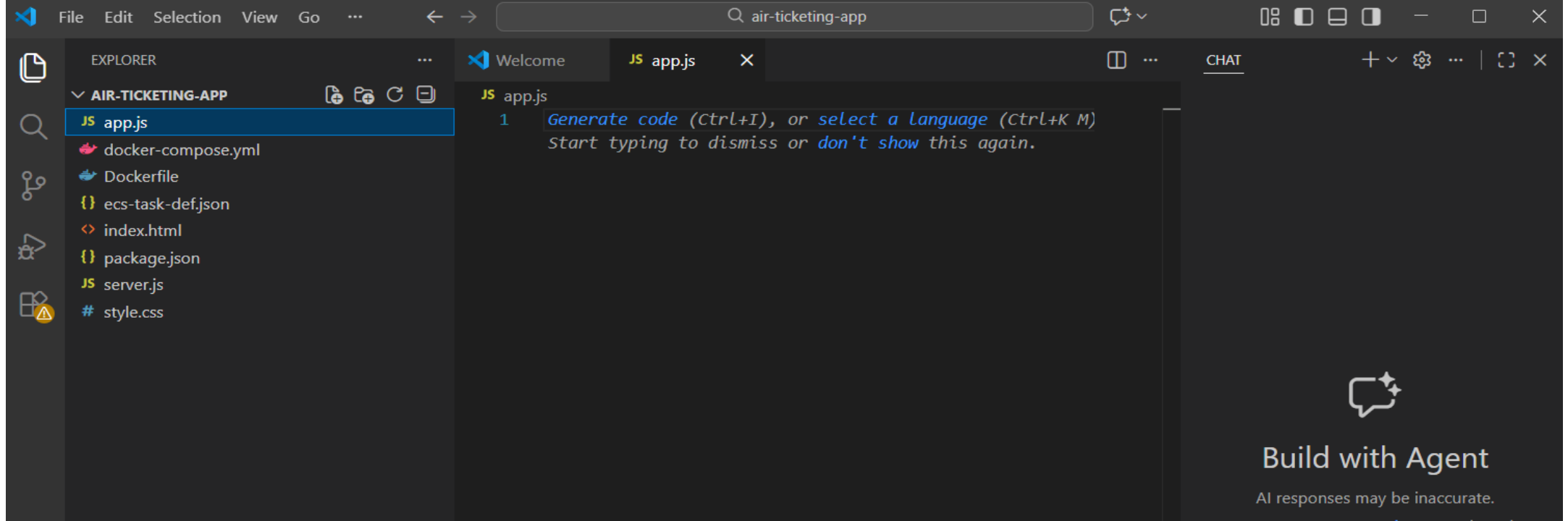
Directory: C:\Users\JAY\air-ticketing-app

Mode                LastWriteTime         Length Name
----                -
-a----            05-02-2026      16:20             0 index.html
-a----            05-02-2026      16:20             0 style.css
-a----            05-02-2026      16:20             0 package.json
-a----            05-02-2026      16:20             0 server.js
-a----            05-02-2026      16:20             0 Dockerfile
-a----            05-02-2026      16:20             0 docker-compose.yml
-a----            05-02-2026      16:20             0 ecs-task-def.json
-a----            05-02-2026      16:20             0 app.js

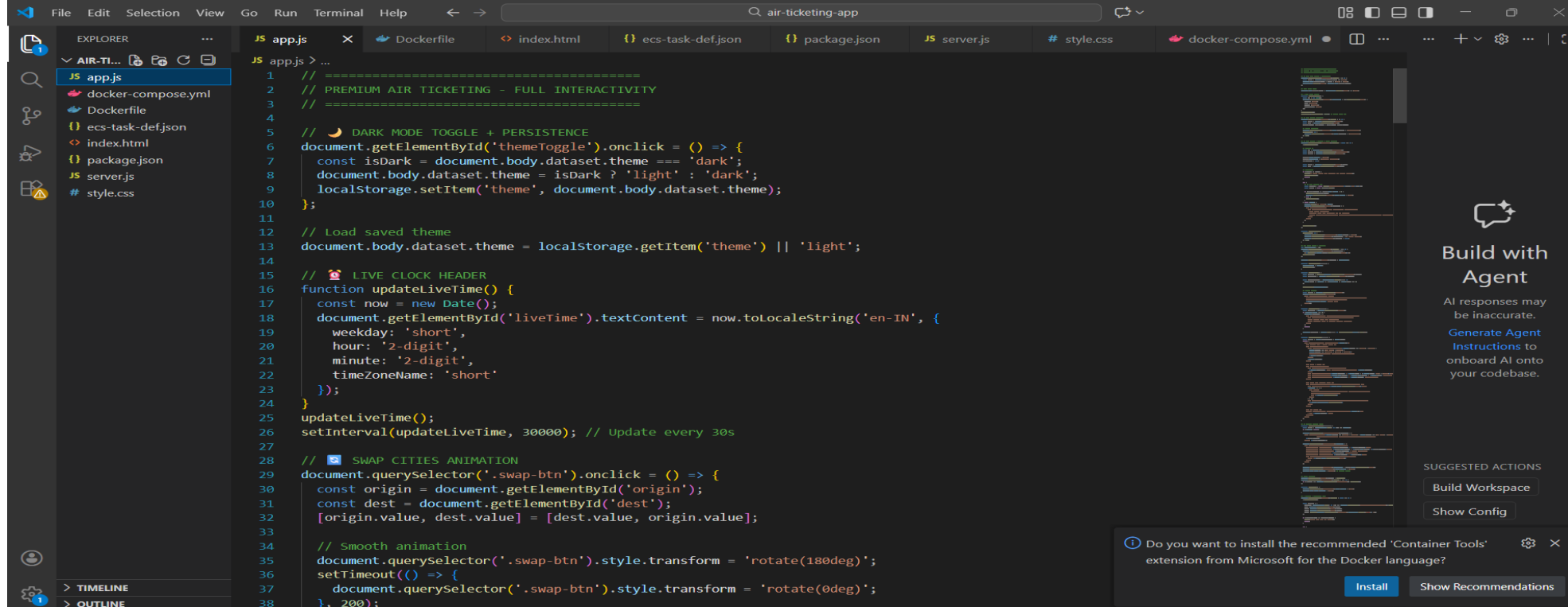
PS C:\Users\JAY\air-ticketing-app>

```

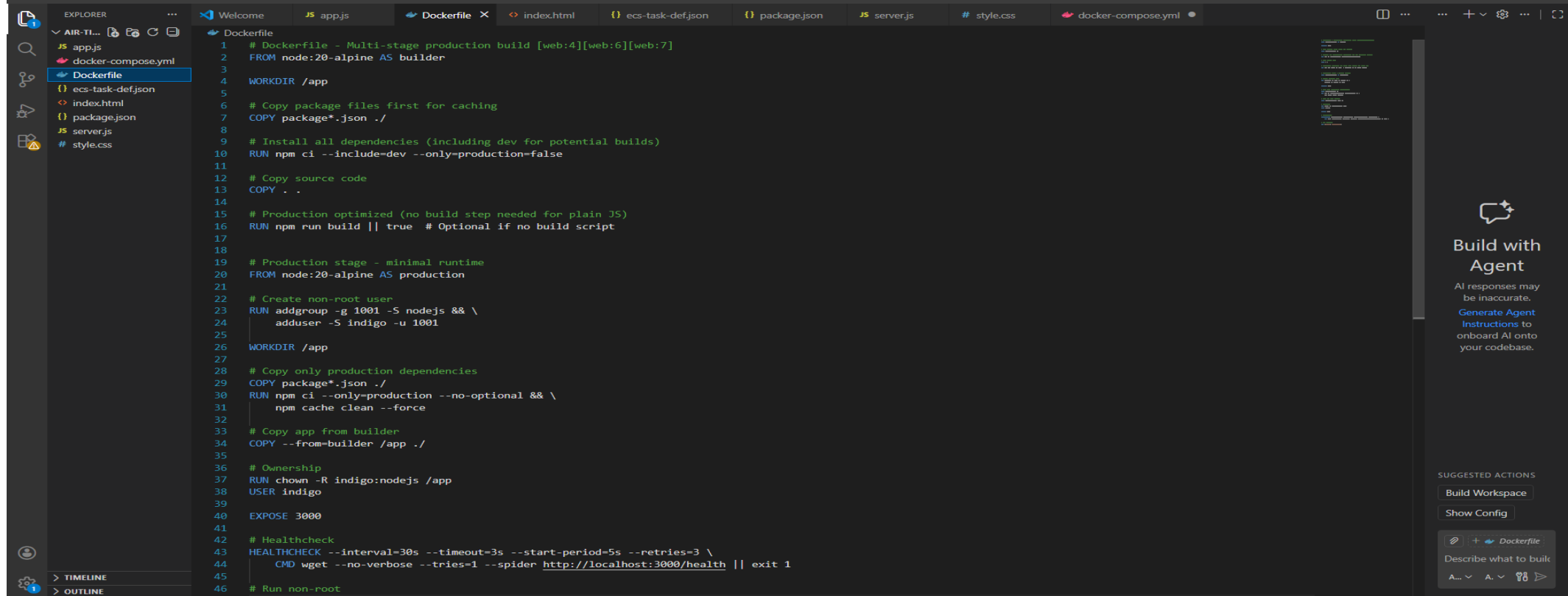
- The command creates multiple files at once: index.html, style.css, package.json, server.js, Dockerfile, docker-compose.yml, ecs-task-def.json, and app.js.
- These files cover front-end, back-end, and Docker/AWS configuration needs.
- The terminal confirms each file was created successfully



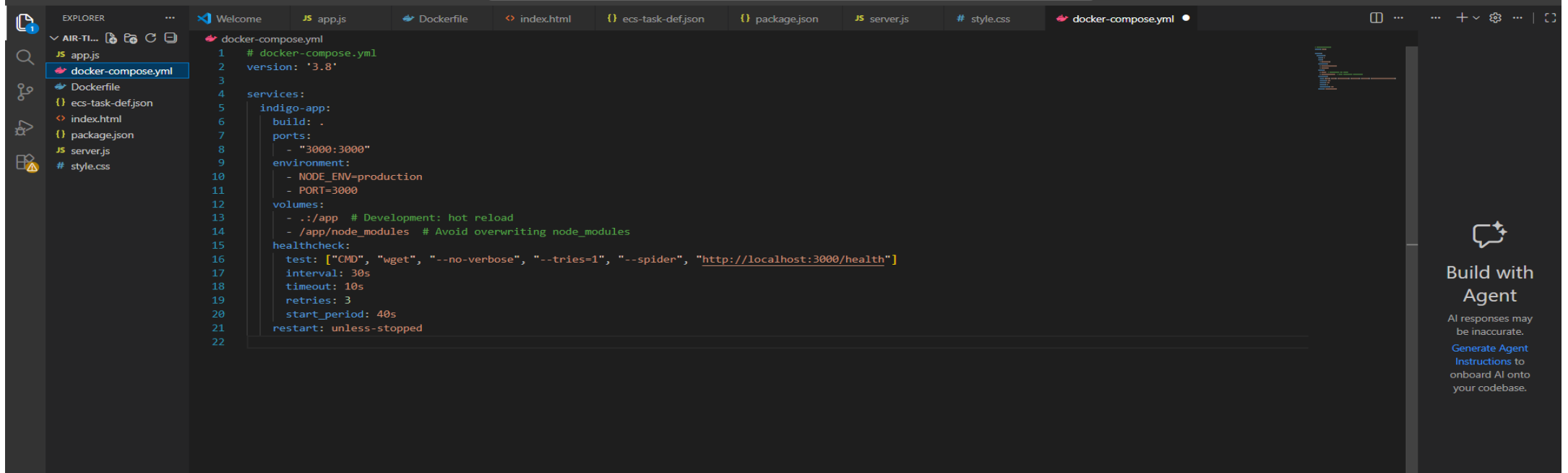
- The left sidebar lists all the files you created earlier: app.js, Dockerfile, docker-compose.yml, ecs-task-def.json, index.html, style.css, package.json, and server.js.



- The code enables dark/light mode toggle and saves the user's theme preference in local storage.
- It displays a live clock header, updating every 30 seconds.
- There's also a swap cities animation, allowing origin and destination inputs to switch smoothly.



- It uses a multi-stage build: first a builder stage installs dependencies and prepares the app, then a production stage creates a lightweight runtime image.
- The builder stage copies package files, installs dependencies, and runs an optional build step.
- The production stage installs only production dependencies, copies the app from the builder, and sets up a non-root user for security.
- It exposes port 3000 and includes a healthcheck to monitor the app's status.



This file is your docker-compose.yml, which defines how your containerized app will run.

- It specifies a service called indigo-app.
- The app runs on port 3000, with environment variables set for production.
- Volumes are mounted to allow hot reload during development and to protect node_modules.


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\JAY\air-ticketing-app> node --version
v24.11.1
PS C:\Users\JAY\air-ticketing-app> 
```

- The command executed is: `node --version`
- The output confirms Node.js version `v24.11.1` is installed.
- This step ensures your environment is ready to run the air-ticketing app and supports Docker builds.

```
PS C:\Users\JAY\air-ticketing-app> node server.js
```

```
✈ IndiGo Air Premium Server  
Port: 3000  
Health: http://localhost:3000/health  
Flights: http://localhost:3000/api/flights?from=BOM&to=DEL  
Static: http://localhost:3000/ (App loads)  
Press Ctrl+C to stop
```

```
PS C:\Users\JAY\air-ticketing-app> █
```

I start it with . Node server.js

- The server runs on port 3000.
- It shows endpoints for health check, flights API, and static content.
- The terminal confirms the server is live.

```
PS C:\Users\JAY\air-ticketing-app> docker --version
Docker version 28.3.0, build 38b7060
PS C:\Users\JAY\air-ticketing-app>
```

I check Docker version with `docker --version`.

- The output shows Docker version 28.3.0, build 38b7060

```
PS C:\Users\JAY\air-ticketing-app> docker compose up --build
time="2026-02-06T16:53:44+05:30" level=warning msg="C:\\Users\\JAY\\air-ticketing-app\\docker-compose.yml: the attribut
avoid potential confusion"
[+] Building 11.1s (13/17)
=> [internal] load local bake definitions
=> => reading from stdin 392B
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.17kB
=> [internal] load metadata for docker.io/library/node:20-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
```

- This starts building the Docker container for my app.
- The process uses the node:20-alpine image and reads instructions from the Dockerfile.
- It loads metadata, transfers context, and sets up the build environment.
- A warning message appears about the docker-compose.yml file, but the build continues.



 Search

[Alt+S] 



Asia Pacific (Mumbai) 

jay2025 (1827-1758-6119)  jay2025



Containers

Amazon Elastic Container Registry

Share and deploy container software, publicly or privately

Amazon Elastic Container Registry (ECR) is a fully managed container registry that makes it easy to store, manage, share, and deploy your container images and artifacts anywhere.

Create a repository

Create

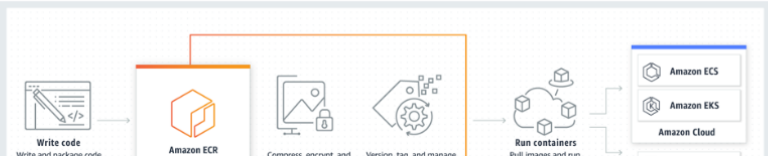
Pricing (US)

You pay only for the amount of data you store in your public or private repositories and data transferred to the Internet.

[ECR pricing](#)

Enable private connection to ECR private repositories using VPC interface endpoints and reduce cost.

How it works



```
graph LR; A[Write code  
Write and markdown code] --> B[Amazon ECR]; B --> C[Compress, encrypt, and  
Version, tag, and manage]; C --> D[Run containers  
Pull images and run]; D --> E[Amazon ECS  
Amazon EKS  
Amazon Cloud];
```

- ECR is a fully managed service to store and manage Docker container images.
- It allows me to share and deploy images either publicly or privately.
- I only pay for the storage and data transfer used.

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

jay2025 (1827-1758-6119)

jay2025

Amazon ECR > Private registry > Repositories > Create private repository

ⓘ 🗨 🔄

Create private repository

General settings

Repository name
Enter a concise name. Repositories support namespaces, which you can use to group similar repositories.

182717586119.dkr.ecr.ap-south-1.amazonaws.com/

12 out of 256 characters maximum (2 minimum). The name must start with a letter and can only contain lowercase letters, numbers, and special characters ._-/.

Image tag settings [Info](#)

Image tag mutability
Choose the tag mutability setting.

☒ **Mutable**
Image tags can be overwritten.

☐ **Immutable**
Image tags can't be overwritten.

Mutable tag exclusions
Tags that match these filters will be immutable (can't be overwritten). Using wildcards (*) will match zero or more image tag characters.

Add filter

Filters must only contain letters, numbers, and special characters (._*-). Each filter is limited to 128 characters, 2 wildcards (*), and you can add up to 5 filters in the exclusion list.

- Repository name: **airline-repo** under my AWS account.
- Image tag mutability is set to **mutable**, so tags can be overwritten.
- There's an option to add filters to make specific tags immutable.
- This setup ensures my Docker images are stored securely and can be managed easily.

aws [Alt+S] Search Asia Pacific (Mumbai) jay2025 (1827-1758-6119) jay2025

Amazon ECR > Private registry > Repositories

Amazon Elastic Container Service

- Private registry
 - Repositories
 - Features & Settings
- Public registry
 - Repositories
 - Settings

ECR public gallery [↗](#)

Amazon ECS [↗](#)

Amazon EKS [↗](#)

Getting started [↗](#)

Documentation [↗](#)

Successfully created private repository, airline-repo

Private repositories (1)

Search by repository substring

	Repository name	URI	Created at	Tag immutability	Encryption type
☐	airline-repo	182717586119.dkr.ecr.ap-south-1.amazonaws.com/airline-repo	February 06, 2026, 17:12:34 (UTC+05.5)	Mutable	AES-256

View push commands Delete Actions Create repository

- Repository name: airline-repo is successfully created.
- The URI is 182717586119.dkr.ecr.ap-south-1.amazonaws.com/airline-repo.
- It shows creation time, tag mutability set to mutable, and encryption type AES-256

```
PS C:\Users\JAY\air-ticketing-app> aws configure
AWS Access Key ID [None]: AKIAI44QH8DHBVS7JL53V4
AWS Secret Access Key [None]: wJalrXU3WhKFb2ZLqWUawENM9WT3T3Zv1v4nW
Default region name [None]: us-east-1
Default output format [None]: json
```

- Using **aws configure** to access aws using access key and secreate key


```
PS C:\Users\JAY\air-ticketing-app> aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 182717586119.dkr.ecr.us-east-1.amazonaws.com
Login Succeeded
```

- The terminal shows Login Succeeded.
- This confirms Docker is authenticated with my private repository.
- Now I can push and pull Docker images securely to AWS ECR.

```

PS C:\Users\JAV\air-ticketing-app> docker build -t indigo-air .
[+] Building 5.2s (17/17) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.17kB
=> [internal] load metadata for docker.io/library/node:20-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 41.16kB
=> [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:09e2b3d9726018aacf269bd35325f46bf75046a643a66d28360ec71132750ec8
=> => resolve docker.io/library/node:20-alpine@sha256:09e2b3d9726018aacf269bd35325f46bf75046a643a66d28360ec71132750ec8
=> CACHED [production 2/7] RUN addgroup -g 1001 -S nodejs && adduser -S indigo -u 1001
=> CACHED [production 3/7] WORKDIR /app
=> CACHED [production 4/7] COPY package*.json ./
=> CACHED [production 5/7] RUN npm ci --only=production --no-optional && npm cache clean --force
=> CACHED [builder 2/6] WORKDIR /app
=> CACHED [builder 3/6] COPY package*.json ./
=> CACHED [builder 4/6] RUN npm ci --include=dev --only=production=false
=> CACHED [builder 5/6] COPY . .
=> CACHED [builder 6/6] RUN npm run build || true # Optional if no build script
=> CACHED [production 6/7] COPY --from=builder /app ./
=> CACHED [production 7/7] RUN chown -R indigo:nodejs /app
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:12b1a6d546eb1522ff0224f37c19d01ced98a0e3c1a033620bf7396c2f581cf3
=> => exporting config sha256:409af3c4cfacc6c61a8e4ea6f8da80805551d1b014c286e2e52bdf9c0221b1c
=> => exporting attestation manifest sha256:6ee9279538e723a83272c1225144fde7dbb7fb85efab391c434bd6fac0eea7d6
=> => exporting manifest list sha256:c6c3b56b92a84f449c7d0ce5d3671f1a27d6ed3e4475fbb658088936745c865e
=> => naming to docker.io/library/indigo-air:latest
=> => unpacking to docker.io/library/indigo-air:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/lhsm66vhrdxsx2jzdx54muk4y

```

- The build runs 17 steps and finishes in 5.2 seconds.
- It uses the node:20-alpine image and loads context from the Dockerfile.
- Most steps are CACHED, speeding up the process.
- The image is tagged as indigo-air:latest and stored locally.
- This confirms my app is containerized and ready to be pushed to AWS ECR

```
PS C:\Users\JAY\air-ticketing-app> docker tag indigo-air:latest 182717586119.dkr.ecr.us-east-1.amazonaws.com/airline-repo:latest
```

- This assigns the indigo-air image to my private repository.
- The tag ensures the image points to airline-repo in AWS ECR.
- Now the image is ready to be pushed to the cloud.
- This step connects my local build with the remote registry for deployment.

```
● PS C:\Users\JAY\air-ticketing-app> docker push 182717586119.dkr.ecr.us-east-1.amazonaws.com/airline-repo:latest
The push refers to repository [182717586119.dkr.ecr.us-east-1.amazonaws.com/airline-repo]
0ba910b04155: Pushed
e31b20165522: Pushed
8f1f30887f2a: Pushed
4e6b615d3716: Pushed
98ff776e181f: Pushed
a98ca684f8f4: Pushed
589002ba0eae: Pushed
eb87f4721c91: Pushed
2ee63dbba2a3: Pushed
6854d099ef57: Pushed
ad6d96c196e3: Pushed
latest: digest: sha256:c6c3b56b92a84f449c7d0ce5d3671f1a27d6ed3e4475fbb658088936745c865e size: 856
○ PS C:\Users\JAY\air-ticketing-app> █
```

- The terminal shows the repository and confirms each image layer is pushed successfully.
- A final digest and size are displayed for the image.
- This means my Docker image is now stored in AWS ECR.
- I can use it directly for deployment with ECS or EKS.

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR

Repositories

AWS Batch

Online learning workshop

Documentation

Discover products

Subscriptions

Tell us what you think

Express Mode

Deploy highly available, scalable, containerized applications with simplified configurations. Express Mode automatically sets up the necessary AWS services and resources.

Let's set up your app

Image URI

182717586119.dkr.ecr.us-east-1.amazonaws.com/airline-repo@sha256:c6c3b56b92a84f449c7d0ce5d3671f1a27d6ed3e4475fb

Browse ECR images

Private registry - optional

☐ Private registry authentication

Task execution role

Create new role

Create new role

A default role ecsTaskExecutionRole will be created automatically. View permission details

Infrastructure role

ecsTaskExecutionRole012

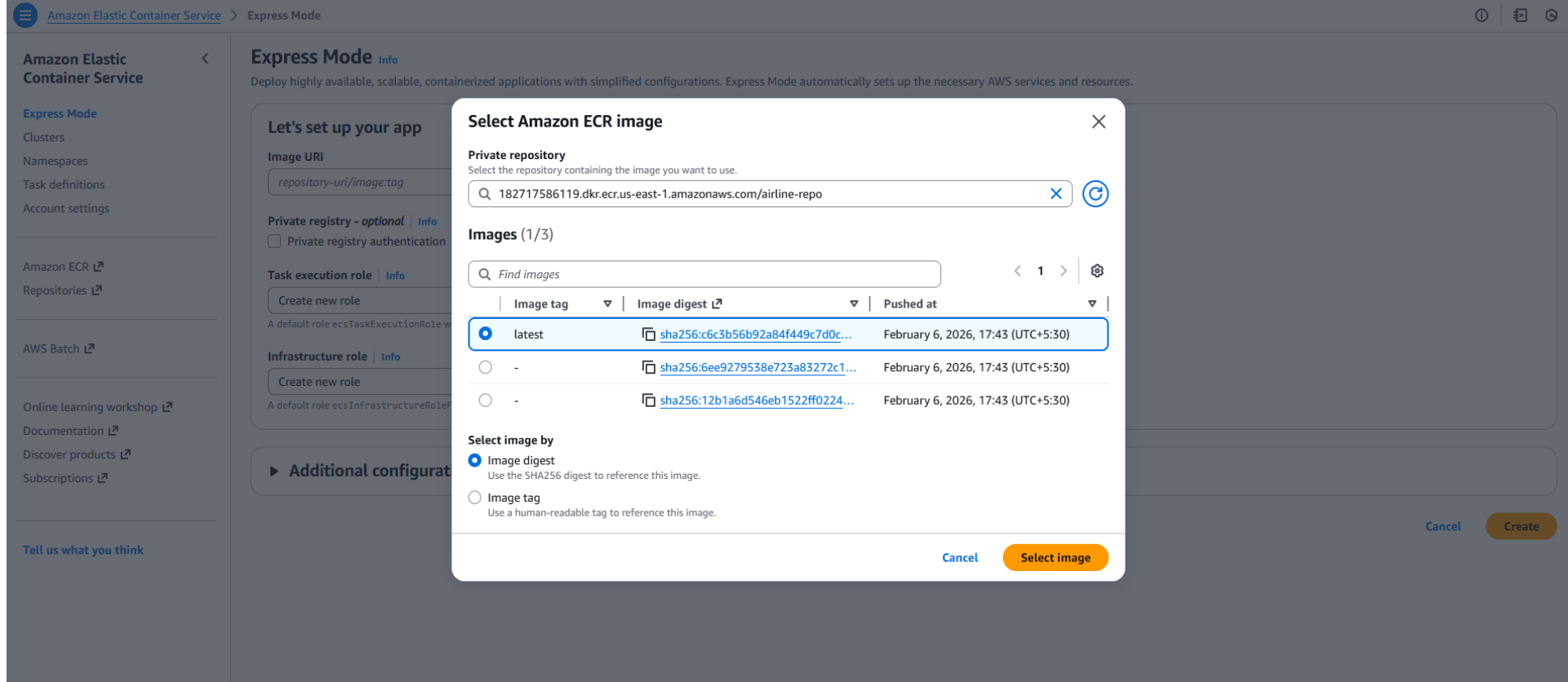
Create new role

Additional configurations - optional

Cancel

Create

- I provide the Image URI from my ECR repository.
- Task execution role is set to create new role (ecsTaskExecutionRole).
- Infrastructure role is selected for ECS to manage resources.
- Private registry authentication is optional but available.
- This step connects my Docker image with ECS, preparing it for deployment.



- In ECS Express Mode, I need to choose which Docker image to deploy.
- The repository airline-repo in AWS ECR contains multiple images with different digests.
- I select the latest tag because it represents the most up-to-date build of my app.
- ECS allows me to reference the image either by digest (unique hash) or by tag.
- Using the digest ensures exact versioning, while tags are easier for continuous updates.
- This step links my container image from ECR to ECS, so ECS can run tasks and services based on it.

aws [Alt+S] Search United States (N. Virginia) jay2025 (1827-1758-6119) jay2025

Amazon Elastic Container Service > Clusters > default > Express services > airline-repo-04eb > Resources

Amazon Elastic Container Service

Express Mode
Clusters
Namespaces
Task definitions
Account settings

Amazon ECR
Repositories

AWS Batch

Online learning workshop
Documentation
Discover products
Subscriptions

Tell us what you think

Successfully created Express service **airline-repo-04eb** in cluster **default**.
A deployment will begin automatically. Your service will be accessible after deployment completes and resources are provisioned. Check the Resources tab to monitor provisioning progress.

[View details](#)

airline-repo-04eb

Express Info

Last updated February 6, 2026, 18:28 (UTC+5:30)

[Delete service](#) [Update service](#)

Express service overview

Service ARN
arn:aws:ecs:us-east-1:182717586119:service/default/airline-repo-04eb

Service status
Active

Tasks
1 Pending | 0 Running [View tasks](#)

Application URL
ai-c9d1a376871b44d58a9e3357ce89ffa2.ecs.us-east-1.on.aws

Active deployment
RSdD-Qe9eHrf4DAI2dTTF is in progress | [View timeline](#)

Updated at
February 6, 2026, 18:23 (UTC+5:30)

Observability | Logs | **Resources** | Deployments | Configuration | Tags

Timeline view | Table view

Resources

Info

Your AWS resources for this service. Updates automatically.

- Cluster: [default](#)
- Task definition: [default-airline-repo-04eb:1](#)
- Target service revision: [6746282473536180585](#)
- Service security group: [sg-0331604090fec8242](#) Active
Last updated at February 6, 2026, 18:24 (UTC+5:30).
- Log group: [/aws/ecs/default/airline-repo-04eb-d660](#) Active
Last updated at February 6, 2026, 18:23 (UTC+5:30).

Last updated February 6, 2026, 18:28 (UTC+5:30)

- The deployment process has begun

aws [Search] [Alt+S]

Amazon Elastic Container Service > Clusters > default > Express services > indigo-air-e698 > Resources

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR

Repositories

AWS Batch

Online learning workshop

Documentation

Discover products

Subscriptions

Tell us what you think

Last updated at February 13, 2026, 17:15 (UTC+5:30).

- ✓ **Public ingress path:** [in-95984bda283a4351a890f7cab2dbb2d7.ecs.us-east-1.on.aws](#) **Active**
Last updated at February 13, 2026, 17:15 (UTC+5:30).
- ✓ **Certificate:** [in-95984bda283a4351a890f7cab2dbb2d7.ecs.us-east-1.on.aws](#) **Active**
Last updated at February 13, 2026, 17:15 (UTC+5:30).
- ✓ **Load balancer security group:** [sg-0ff6c6373b0a8a655](#) **Active**
Last updated at February 13, 2026, 17:15 (UTC+5:30).
- ✓ **Load balancer:** [app/ecs-express-gateway-alb-52920bc4/93896cebc5f3dfac](#) **Active**
Last updated at February 13, 2026, 17:15 (UTC+5:30).
- ✓ **Listener:** [app/ecs-express-gateway-alb-52920bc4/93896cebc5f3dfac/3265bfb1b63bc266](#) **Active**
Last updated at February 13, 2026, 17:15 (UTC+5:30).
- ✓ **Target group:** [ecs-gateway-tg-61441a4542fd8a69f/c26fdeb862d3d090](#) **Active**
Last updated at February 13, 2026, 17:15 (UTC+5:30).
- ✓ **Target group:** [ecs-gateway-tg-3515d4955271d960e/9e4afd6e4c1ee8db](#) **Active**
Last updated at February 13, 2026, 17:15 (UTC+5:30).
- ✓ **Listener rule:** [app/ecs-express-gateway-alb-52920bc4/93896cebc5f3dfac/3265bfb1b63bc266/690cb194df1cc72b](#) **Active**
Last updated at February 13, 2026, 17:15 (UTC+5:30).
- ✓ **Scaling target:** [service-default-indigo-air-e698](#) **Active**
Last updated at February 13, 2026, 17:12 (UTC+5:30).
- ✓ **Scaling policy:** [indigo-air-e698](#) **Active**
Last updated at February 13, 2026, 17:12 (UTC+5:30).
- ✓ **Service:** [indigo-air-e698](#) **Active**
service [indigo-air-e698](#) has reached a steady state.
- ✓ **Deployment** [8aQ9eZHBvnpslicVEk9kC](#) **has completed.**
Completed at February 13, 2026, 17:17 (UTC+5:30)

- My website has been created successfully.

aws [Search] [Alt+S] United States (N. Virginia) jay2025 (1827-1758-6119) jay2025

Amazon Elastic Container Service > Clusters > default > Express services > indigo-air-e698 > Resources

Amazon Elastic Container Service

Express Mode
Clusters
Namespaces
Task definitions
Account settings

Amazon ECR
Repositories

AWS Batch

Online learning workshop
Documentation
Discover products
Subscriptions

Successfully created Express service indigo-air-e698 in cluster default
A deployment will begin automatically. Your service will be accessible after deployment completes and resources are provisioned. Check the Resources tab to monitor provisioning progress. [View details](#)

indigo-air-e698 Express Info Last updated February 13, 2026, 17:18 (UTC+5:30) [Delete service](#) [Update service](#)

Express service overview

Service ARN
arn:aws:ecs:us-east-1:182717586119:service/default/indigo-air-e698

Service status
Active

Tasks
0 Pending | 1 Running [View tasks](#)

Application URL
in-95984bda283a4351a890f7cab2dbb2d7.ecs.us-east-1.on.aws

Active deployment
-

Updated at
February 13, 2026, 17:12 (UTC+5:30)

Observability | Logs | **Resources** | Deployments | Configuration | Tags

Timeline view Table view

Resources Info Last updated February 13, 2026, 17:18 (UTC+5:30)

Your AWS resources for this service. Updates automatically.

- Using this url to access website



Book Flights Across India

FROM

 Departure



TO

 Arrival

DEPART

dd-mm-yyyy 

 **SEARCH FLIGHTS**

- The website was successfully deployed.